

NAME:R.KAVIYA

REG NO:192524154

COURSE:CSA0904-PROGRAMMING IN JAVA

1. ATM PIN Security Validator

A nationalized bank receives ATM PINs from customers through a kiosk.

Develop a Java program that accepts the PIN as a string and performs the

following operations:

Remove unwanted spaces using trim()

Validate that the PIN contains exactly 4 digits using matches()

Display the PIN length using length();

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Enter PIN: ");
8         String pin = sc.nextLine().trim();
9
10        System.out.println("PIN Length: " + pin.length());
11
12        if (pin.matches("\\d{4}"))
13            System.out.println("Valid PIN");
14        else
15            System.out.println("Invalid PIN");
16    }
17 }
```

```
PIN Length: 4
Valid PIN
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

2. Employee ID Normalization System

An HR department receives employee IDs in mixed formats such as emp1023.

Create a Java program to:

Remove spaces using trim()

Convert the alphabetic prefix to uppercase using toUpperCase()

Join the prefix and number using concat()

```
1 import java.util.*;
2
3 public class Main{
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Enter Employee ID: ");
8         String id = sc.nextLine().trim();
9
10        String prefix = id.replaceAll("[0-9]", "").toUpperCase();
11        String number = id.replaceAll("[A-Za-z]", "");
12
13        System.out.println("Normalized ID: " + prefix.concat(number));
14    }
15 }
```

Enter Employee ID: emp1023
Normalized ID: EMP1023

...Program finished with exit code 0
Press ENTER to exit console.

3. Online Shopping Coupon Verification

An e-commerce website offers the coupon code SAVE50. Write a Java program that:

Removes extra spaces using trim()

Converts the entered code to uppercase using toUpperCase()

Compares it with the original coupon using equalsIgnoreCase()

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String original = "SAVE50";
8
9         System.out.print("Enter Coupon: ");
10        String code = sc.nextLine().trim().toUpperCase();
11
12        if (code.equalsIgnoreCase(original))
13            System.out.println("Coupon Accepted");
14        else
15            System.out.println("Invalid Coupon");
16    }
17 }
```

Enter Coupon: save50
Coupon Accepted

...Program finished with exit code 0
Press ENTER to exit console.

4. Student Email Username Extractor

A college portal stores student emails such as arun23@college.edu.

Develop a program that:

Finds the position of @ using indexOf()

Extracts the username using substring()

Displays the username length using length

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Enter Email: ");
8         String email = sc.nextLine();
9
10        int pos = email.indexOf("@");
11        String user = email.substring(0, pos);
12
13        System.out.println("Username: " + user);
14        System.out.println("Length: " + user.length());
15    }
16 }
```

```
Enter Email: kaviya1217@college.edu
Username: kaviya1217
Length: 10
```

```
...Program finished with exit code 0
```

5. Restaurant Digital Order Analyzer

A restaurant stores orders as DOSA-3-IDLY-2-VADA-1. Create a program that:

Splits the order using `split()`

Converts food names to uppercase using `toUpperCase()`

Counts the number of ordered items using `length()`

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Enter Order: ");
8         String order = sc.nextLine();
9
10        String[] a = order.split("-");
11
12        System.out.println("Food Items:");
13        for (int i = 0; i < a.length; i += 2)
14            System.out.println(a[i].toUpperCase());
15
16        System.out.println("Number of Items: " + a.length / 2);
17    }
18 }
```

```
Enter Order: dosa-3-idly-2-dosa-1
Food Items:
DOSA
IDLY
DOSA
Number of Items: 3
```

6. Railway Coach and Seat Parser

Indian Railways stores ticket information as S5-43. Write a Java program that:

Splits the value using `split()`

Extracts the coach number using `substring()`

Displays the coach type using `charAt()`

```
1- import java.util.*;
2
3- public class Main {
4-     public static void main(String[] args) {
5-         Scanner sc = new Scanner(System.in);
6
7-         System.out.print("Enter Ticket: ");
8-         String ticket = sc.nextLine();
9
10        String[] a = ticket.split("-");
11
12        System.out.println("Coach: " + a[0]);
13        System.out.println("Coach Number: " + a[0].substring(1));
14        System.out.println("Coach Type: " + a[0].charAt(0));
15    }
16 }
```

Enter Ticket: 55-43

Coach: 55

Coach Number: 5

Coach Type: 5

...Program finished with exit code 0

7. Inventory Product Search System

A supermarket stores product names in different letter cases. Develop a program that:

Converts product names to lowercase using `toLowerCase()`

Searches a product using `contains()`

Confirms exact matching using `equalsIgnoreCase`

```
1- import java.util.*;
2
3- public class Main {
4-     public static void main(String[] args) {
5-         Scanner sc = new Scanner(System.in);
6
7-         String product = "Rice";
8
9-         System.out.print("Enter Product: ");
10        String p = sc.nextLine().toLowerCase();
11
12        System.out.println("Contains: " + product.toLowerCase().contains(p));
13
14        if (product.equalsIgnoreCase(p))
15            System.out.println("Exact Match");
16        else
17            System.out.println("Not Exact Match");
18    }
19 }
```

Enter Product: RICE
Contains: true
Exact Match

...Program finished with exit code 0
Press ENTER to exit console.

8. Grocery Shopping Cart Management System

Add products using add()

Remove purchased products using remove()

Check whether a product exists using contains() Check how many products

there in the list

```
1- import java.util.*;
2
3- public class Main {
4-     public static void main(String[] args) {
5
6         ArrayList<String> list=new ArrayList<>();
7
8         list.add("Rice");
9         list.add("Oil");
10        list.add("Sugar");
11
12        list.remove("Oil");
13
14        System.out.println(list);
15
16        System.out.println("Contains Rice: "+list.contains("Rice"));
17
18        System.out.println("Total Products: "+list.size());
19    }
20 }
```

[Rice, Sugar]

Contains Rice: true

Total Products: 2

...Program finished with exit code 0

Press ENTER to exit console.

9. Bank Token Queue Management System

A bank serves customers based on token order. Using Queue:

Add customer tokens using offer()

Serve the next customer using poll()

Display the upcoming customer using peek()

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Queue<Integer> q=new LinkedList<>();
7
8         q.offer(101);
9         q.offer(102);
10        q.offer(103);
11
12        System.out.println("Served: "+q.poll());
13
14        System.out.println("Next: "+q.peek());
15
16        System.out.println(q);
17    }
18 }
```

Served: 101

Next: 102

[102, 103]

...Program finished with exit code 0

Press ENTER to exit console.

10. Railway Parcel Dispatch Tracking System

A railway parcel office loads parcels onto a dispatch container. The last loaded parcel must be unloaded first, and parcel details must be searchable.

Requirements:

Store parcel IDs in a Stack

Store parcel ID and destination in a HashMap

Unload parcels using pop() and display the destination of the unloaded parcel

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Stack<String> stack=new Stack<>();
7         HashMap<String,String> map=new HashMap<>();
8
9         stack.push("P101");
10        stack.push("P102");
11
12        map.put("P101","Chennai");
13        map.put("P102","Madurai");
14
15        String id=stack.pop();
16
17        System.out.println("Unloaded: "+id);
18        System.out.println("Destination: "+map.get(id));
19    }
20 }
```

```
Unloaded: P102
Destination: Madurai
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

11. Hospital Appointment Scheduling System

A hospital issues appointment tokens to patients and stores patient details separately.

Requirements:

Add appointment tokens to a Queue

Store token number and patient name in a HashMap

Serve patients in token order and display the served patient details

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Queue<Integer> q=new LinkedList<>();
7         HashMap<Integer,String> map=new HashMap<>();
8
9         q.offer(1);
10        q.offer(2);
11
12        map.put(1,"Arun");
13        map.put(2,"Bala");
14
15        int token=q.poll();
16
17        System.out.println("Served Token: "+token);
18        System.out.println("Patient: "+map.get(token));
19    }
20 }
```

```
Served Token: 1
Patient: Arun
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

12. University Course Enrollment Manager

A university maintains a list of students enrolled in a course. Students who withdraw must be removed from the list.

Requirements:

Add student names to an ArrayList

Traverse the list using an Iterator

Remove withdrawn students and display the updated enrollment list

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         ArrayList<String> list=new ArrayList<>();
7
8         list.add("yuva");
9         list.add("kavi");
10        list.add("anbu");
11
12        Iterator<String> it=list.iterator();
13
14        while(it.hasNext()){
15            if(it.next().equals("Bala"))
16                it.remove();
17        }
18
19        System.out.println(list);
20    }
21 }
```

[yuva, kavi, anbu]

...Program finished with exit code 0
Press ENTER to exit console.

13. Warehouse Damaged Item Removal

"A warehouse stores item names in an inventory list. Damaged items must be removed safely while traversing the collection.

Requirements:

Store inventory items in an ArrayList

Traverse the list using an Iterator

Remove items marked as Damaged and display the remaining inventory

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         ArrayList<String> list=new ArrayList<>();
7
8         list.add("Rice");
9         list.add("Damaged");
10        list.add("Sugar");
11
12        Iterator<String> it=list.iterator();
13
14        while(it.hasNext()){
15            if(it.next().equals("Damaged"))
16                it.remove();
17        }
18
19        System.out.println(list);
20    }
21 }
```

[Rice, Sugar]

...Program finished with exit code 0
Press ENTER to exit console.

14. Bank Token Management System

Customers receive tokens in the order they arrive at a bank counter.

Requirements:

Add customer tokens to a Queue

Serve customers in FIFO order

Display the next customer to be served and the remaining queue

```
Main.java :
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Queue<Integer> q=new LinkedList<>();
7
8         q.offer(1);
9         q.offer(2);
10        q.offer(3);
11
12        System.out.println("Served: "+q.poll());
13
14        System.out.println("Next: "+q.peek());
15
16        System.out.println("Remaining: "+q);
17    }
18 }

Served: 1
Next: 2
Remaining: [2, 3]

...Program finished with exit code 0
Press ENTER to exit console.
```

15. Web Browser Back Navigation

A web browser stores recently visited pages so that users can navigate backward.

Requirements:

Add visited page URLs using push()

Perform the Back operation using pop()

Display the current page using peek()\

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Stack<String> s=new Stack<>();
7
8         s.push("Google");
9         s.push("YouTube");
10        s.push("OpenAI");
11
12        System.out.println("Back: "+s.pop());
13
14        System.out.println("Current Page: "+s.peek());
15    }
}
```

input

Back: OpenAI
Current Page: YouTube

...Program finished with exit code 0
Press ENTER to exit console.